Visual Product Search Engine
AID-825 Visual Recognition Course Project

Sahil Ajaykumar Kolte
IMT2023066

Pratham Arun Shetty
IMT2023534

Lakshya Sachan
IMT2023612

15 May 2026

Abstract

Traditional product search systems depend heavily on text queries and manually written metadata. In fashion retrieval, this often becomes unreliable because users usually search visually rather than verbally. Describing clothing accurately using keywords is difficult, and product descriptions across catalogs are often inconsistent.

In this project, we build a query-by-image fashion retrieval system that allows a user to upload a clothing image and retrieve visually and semantically similar products from a catalog. Our pipeline combines YOLOv8 for clothing localization, BLIP-2 for semantic caption generation, fine-tuned CLIP embeddings for representation learning, and FAISS HNSW for efficient nearest-neighbor search.

We perform an ablation study across multiple retrieval configurations and evaluate using Recall@K, NDCG@K, and mAP@K for K $\in \{5, 10, 15\}$. Our best-performing system uses fine-tuned CLIP embeddings combined with BLIP-generated captions using fusion weight $\alpha = 0.7$. The final system achieves strong retrieval performance while remaining efficient enough for interactive usage.

Contents

1 Code Availability

All source code, evaluation scripts, Streamlit application files, and implementation details referenced in this report are available in the project GitHub repository:

`https://github.com/Pratham007000/VR-Final-Project`

The repository also contains the metric computation scripts (`evaluate.py` and `demo_eval.py`) used for computing Recall@K, NDCG@K, and mAP@K during retrieval evaluation.

2 Problem Statement and Motivation

Online shopping platforms usually rely on keyword-based search. However, users often think visually rather than textually. A person may want to search for "a blue oversized hoodie similar to this image" without knowing the exact product name or category.

This creates a gap between how users think about clothing and how products are stored inside catalogs. Text metadata is often incomplete, inconsistent, or noisy. Because of this, keyword search alone is not sufficient for many fashion retrieval scenarios.

To address this problem, we build a visual product search engine where users can upload a query image and retrieve visually similar products directly from a catalog. The system combines visual understanding, semantic reasoning, and efficient retrieval techniques to perform instance-level clothing retrieval.

Our pipeline is designed in two phases:

- Offline indexing pipeline
- Online inference and retrieval pipeline

The offline stage processes and indexes gallery images once, while the online stage handles user queries in near real time.

3 Dataset

We use the DeepFashion In-Shop Clothes Retrieval dataset for all experiments.

The dataset contains:

- Clothing images from multiple viewpoints
- Product-level item IDs
- Bounding box annotations
- Official train/query/gallery splits

The dataset is divided into three groups:

- **Train Split:** Used for fine-tuning CLIP embeddings
- **Gallery Split:** Used to build the searchable FAISS index
- **Query Split:** Used for retrieval evaluation

A retrieval result is considered correct only if the retrieved image shares the same `item_id` as the query image.

4 System Overview

The complete system contains two major stages:

1. Offline indexing pipeline
2. Online retrieval pipeline

The offline stage converts gallery images into compact embeddings and stores them inside a FAISS index. During inference, the query image is processed using the same embedding pipeline and nearest neighbors are retrieved from the index.

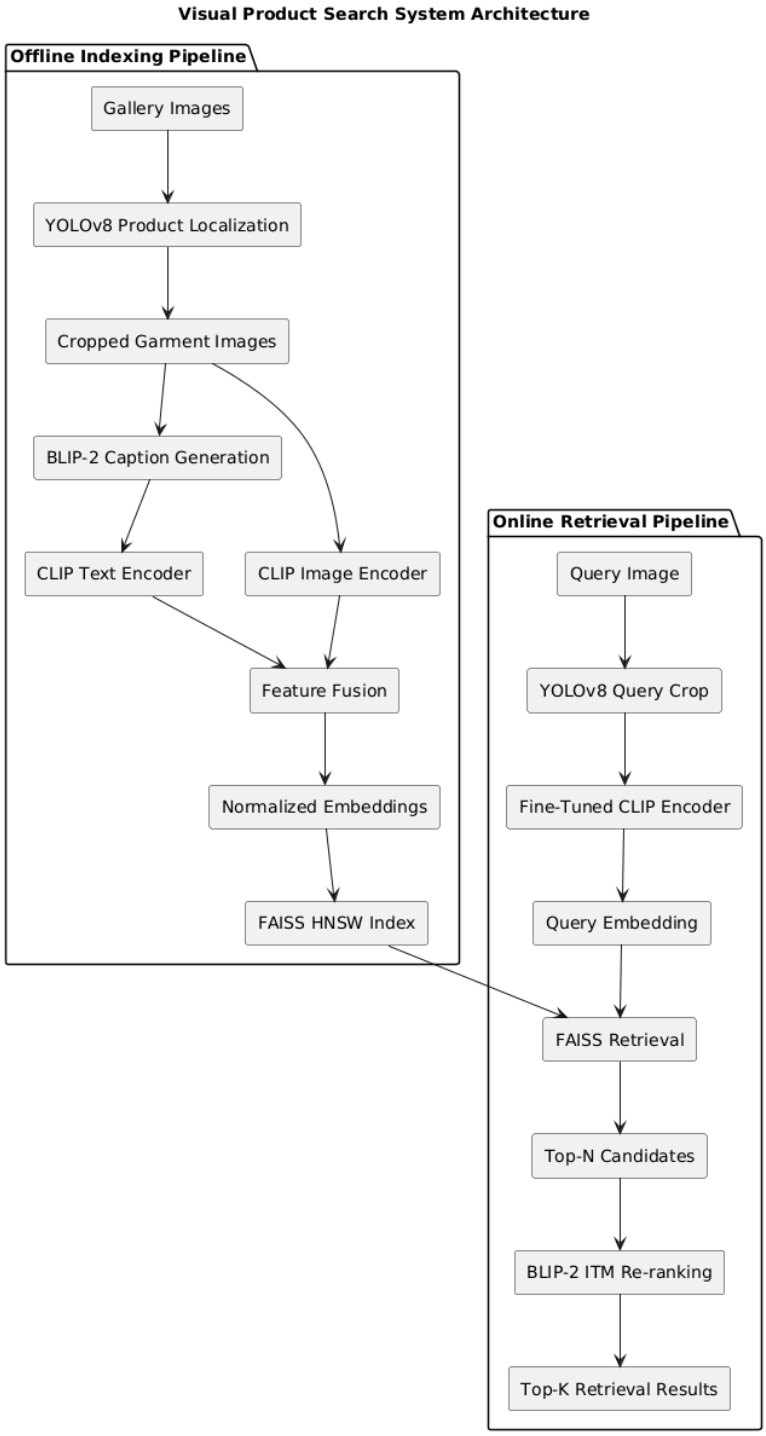

Figure 1: Overall system architecture.

5 Offline Indexing Pipeline

The offline pipeline processes all gallery images before inference begins.

5.1 Step 1: Product Localization using YOLOv8

Raw catalog images usually contain unnecessary background information such as models, studio backgrounds, accessories, or multiple garments.

To reduce noise, we use YOLOv8 to detect the main clothing region and crop the garment before feature extraction.

The detector is trained using DeepFashion bounding box annotations and predicts three broad categories:

- Upper-body clothing
- Lower-body clothing
- Full-body clothing

Only the highest-confidence clothing crop is passed to the downstream retrieval pipeline.

5.2 Step 2: Semantic Caption Generation using BLIP-2

Visual features alone sometimes fail to capture semantic information such as clothing style, texture, or fit.

To improve semantic understanding, we generate captions for each cropped garment using BLIP-2. The generated captions describe clothing attributes such as:

- Color
- Texture
- Clothing type
- Style details

An example caption generated by BLIP-2 is:

> "a blue denim jacket with front buttons and full sleeves"

These captions provide additional semantic information which helps improve retrieval quality.

5.3 Step 3: CLIP Embedding Generation

We use CLIP to encode both visual and textual information into a shared embedding space.

For each gallery image:

- The cropped garment image is passed through the CLIP image encoder
- The generated BLIP caption is passed through the CLIP text encoder

The two embeddings are fused using weighted averaging:

$$v_i = \frac{\alpha \phi_V(x_i) + (1-\alpha)\phi_T(c_i)}{\|\alpha \phi_V(x_i) + (1-\alpha)\phi_T(c_i)\|_2} \tag{1}$$

where:

- ϕ_V is the CLIP image encoder
- ϕ_T is the CLIP text encoder
- x_i is the cropped image
- c_i is the generated caption
- α controls image-text fusion strength

We experiment with:

- $\alpha = 1.0$ (vision only)
- $\alpha = 0.7$
- $\alpha = 0.5$

Our experiments show that $\alpha = 0.7$ gives the best retrieval performance.

5.4 Step 4: FAISS HNSW Indexing

The final fused embeddings are stored inside a FAISS HNSW index for fast nearest-neighbor retrieval.

FAISS allows efficient similarity search even with large gallery sizes. Instead of performing brute-force comparison against every gallery image, HNSW builds a graph-based structure that supports sub-linear retrieval time.

Each index entry stores:

- Fused embedding vector
- Image path
- Item ID
- Clothing category metadata

5.5 HNSW Configuration

We use cosine similarity over L2-normalized embeddings. The FAISS HNSW index is configured using:

- Graph degree parameter: $M = 32$
- Construction parameter: `efConstruction` $= 200$
- Search parameter: `efSearch` $= 128$

These settings provide a good trade-off between retrieval accuracy and inference latency.

6 Training and Fine-Tuning Strategy

We initialize all models using publicly available pretrained checkpoints.

- YOLOv8 is used for clothing localization
- BLIP-2 is used for caption generation and reranking
- CLIP is used for multimodal embedding generation

The main training step in our project is fine-tuning the CLIP vision encoder.

6.1 Contrastive Fine-Tuning

The original CLIP model is trained for general image-text alignment and is not specifically optimized for instance-level product retrieval.

To improve retrieval performance, we fine-tune the CLIP vision encoder using contrastive learning.

Training objective:

- Images with the same `item_id` are treated as positive pairs
- Images with different `item_id`s are treated as negative pairs

This training encourages embeddings of the same product to move closer together while separating unrelated products.

The CLIP text encoder remains frozen during fine-tuning.

6.2 Training Seeds

To ensure reproducibility, experiments were repeated using three different random seeds:

- 2023066
- 2023534
- 2023612

Results are reported as mean ± standard deviation across these runs.

The best-performing checkpoint used in our final system is:

`clip_finetuned_seed2023612.pt`

7 Evaluation Protocol

We evaluate retrieval performance using three standard retrieval metrics.

All metrics are computed at:

$$K \in \{5, 10, 15\}$$

7.1 Recall@K

Recall@K measures whether at least one correct result appears within the top-K retrieved items.

7.2 NDCG@K

Normalized Discounted Cumulative Gain (NDCG) is a ranking-aware metric which gives higher importance to relevant items appearing earlier in the ranking.

7.3 mAP@K

Mean Average Precision measures both retrieval correctness and ranking quality.

Higher values for all three metrics indicate better retrieval performance.

8 Experimental Results and Ablation Study

We perform an ablation study to evaluate the contribution of each component in the retrieval pipeline.

Three major configurations are tested:

- **Config A:** Vision-only CLIP baseline
- **Config B:** Frozen CLIP + BLIP captions
- **Config C:** Fine-tuned CLIP + BLIP captions

The final reported values are averaged across three random seeds.

Config	α	R@5	N@5	mAP@5	R@10	N@10	mAP@10	R@15	N@15	mAP@15
A (Vision-only)	1.0	0.3658 ± 0.0000	0.2918 ± 0.0000	0.2674 ± 0.0000	0.4373 ± 0.0000	0.3150 ± 0.0000	0.2770 ± 0.0000	0.4816 ± 0.0000	0.3267 ± 0.0000	0.2804 ± 0.0000
B (Frozen+BLIP)	0.7	0.3617 ± 0.0000	0.2898 ± 0.0000	0.2659 ± 0.0000	0.4352 ± 0.0000	0.3136 ± 0.0000	0.2758 ± 0.0000	0.4781 ± 0.0000	0.3250 ± 0.0000	0.2792 ± 0.0000
B (Frozen+BLIP)	0.5	0.3534 ± 0.0000	0.2815 ± 0.0000	0.2576 ± 0.0000	0.4276 ± 0.0000	0.3055 ± 0.0000	0.2676 ± 0.0000	0.4702 ± 0.0000	0.3168 ± 0.0000	0.2710 ± 0.0000
C (FT+BLIP)	0.7	0.7063 ± 0.0595	0.6017 ± 0.0679	0.5669 ± 0.0706	0.7867 ± 0.0480	0.6279 ± 0.0642	0.5778 ± 0.0691	0.8265 ± 0.0398	0.6385 ± 0.0620	0.5810 ± 0.0684
C (FT+BLIP)	0.5	0.7055 ± 0.0587	0.5996 ± 0.0659	0.5642 ± 0.0682	0.7856 ± 0.0473	0.6256 ± 0.0622	0.5751 ± 0.0667	0.8262 ± 0.0394	0.6364 ± 0.0601	0.5783 ± 0.0660

Table 1: Ablation study results reported as mean ± standard deviation across multiple random seeds.

8.1 Observations

The ablation study shows that fine-tuning CLIP contributes the largest improvement in retrieval quality.

Compared to the frozen baseline, fine-tuned CLIP significantly improves:

- Recall
- Ranking quality
- Retrieval precision

Adding BLIP-generated captions also improves semantic understanding and helps the model retrieve visually similar items more consistently.

The best-performing setup is:

$$\textbf{Config C (Fine-Tuned CLIP + BLIP)}, \alpha = 0.7$$

This configuration achieves the highest scores across all retrieval metrics. Compared to the vision-only baseline (Config A), the best-performing configuration (Config C, $\alpha = 0.7$) improves Recall@5 from 0.3658 to 0.7063, corresponding to an absolute improvement of 34.05 percentage points.

Similarly:

- NDCG@10 improves from 0.3150 to 0.6279
- mAP@15 improves from 0.2804 to 0.5810

These results demonstrate that CLIP fine-tuning contributes substantially more to retrieval quality than caption fusion alone.

Interestingly, adding BLIP-generated captions without CLIP fine-tuning (Config B) does not improve performance over the vision-only baseline. This suggests that the frozen CLIP embedding space is not sufficiently adapted for instance-level fashion retrieval.

As a result, the semantic information from BLIP captions may introduce additional noise instead of improving alignment between visually similar products.

After fine-tuning CLIP on DeepFashion item identities, the embedding space becomes significantly more discriminative. In this setting, caption-based semantic information becomes much more useful, leading to the large retrieval improvements observed in Config C.

8.2 Fair Experimental Setup

To ensure fair comparison across all configurations, we keep the following components fixed throughout all experiments:

- Same DeepFashion query/gallery split
- Same FAISS HNSW index configuration
- Same retrieval metric implementation
- Same query preprocessing pipeline
- Same evaluation protocol and top-K values

The only variables changed during ablation are:

- CLIP fine-tuning
- Image-text fusion weight α
- Use of BLIP-generated captions

This ensures that observed improvements are directly attributable to the modified retrieval components.

9 Batch Evaluation Demo Script

To satisfy the project evaluation requirements, we implemented a complete batch evaluation script named:

`demo_eval.py`

The script accepts a folder of query images and performs:

1. YOLOv8 product localization
2. Fine-tuned CLIP embedding generation
3. FAISS nearest-neighbor retrieval

4. BLIP-2 semantic reranking
5. Automatic metric computation

The script computes:

- Recall@5/10/15
- NDCG@5/10/15
- mAP@5/10/15

using the best-performing retrieval configuration.

The inference pipeline clearly documents which model is responsible for each stage of the system.

9.1 Best Model Used in Demo Script

- Config C (FT+BLIP)
- Fusion weight $\alpha = 0.7$
- Checkpoint: `clip_finetuned_seed2023612.pt`

9.2 Metric Computation and Evaluation Functions

The retrieval metrics used in this project are implemented inside `evaluate.py` and `demo_eval.py`. These scripts are responsible for evaluating retrieval quality on the DeepFashion query/gallery split and computing all project metrics automatically.

The evaluation pipeline processes each query image, retrieves the top-K nearest neighbors from the FAISS index, and compares the retrieved item IDs with the ground-truth item ID of the query image.

The major metric computation functions implemented in the evaluation scripts are:

- **Recall@K:** Checks whether at least one correct item appears within the top-K retrieved results.
- **NDCG@K:** Computes ranking-aware retrieval quality by assigning higher importance to correct items appearing earlier in the ranked list.
- **mAP@K:** Computes Mean Average Precision over the top-K retrieved items by considering both retrieval correctness and ranking order.

The `evaluate.py` script is primarily used for quantitative evaluation over the dataset query split, while `demo_eval.py` provides automated end-to-end batch evaluation for folders of query images during project demonstration.

The implemented evaluation functions ensure that retrieval performance is measured consistently across all experimental configurations and ablation studies.

10 Online Inference Pipeline

We also developed an interactive Streamlit application for real-time retrieval.

The application allows users to:

- Upload query images

- Run retrieval interactively
- View top-K similar products
- Inspect retrieval scores

10.1 Step 1: Query Upload

The user uploads a query image through the Streamlit interface.Before retrieval begins, the Streamlit application displays the detected crop and asks the user to either confirm the detection or upload a new query image. This step ensures that incorrect detections do not propagate through the retrieval pipeline.

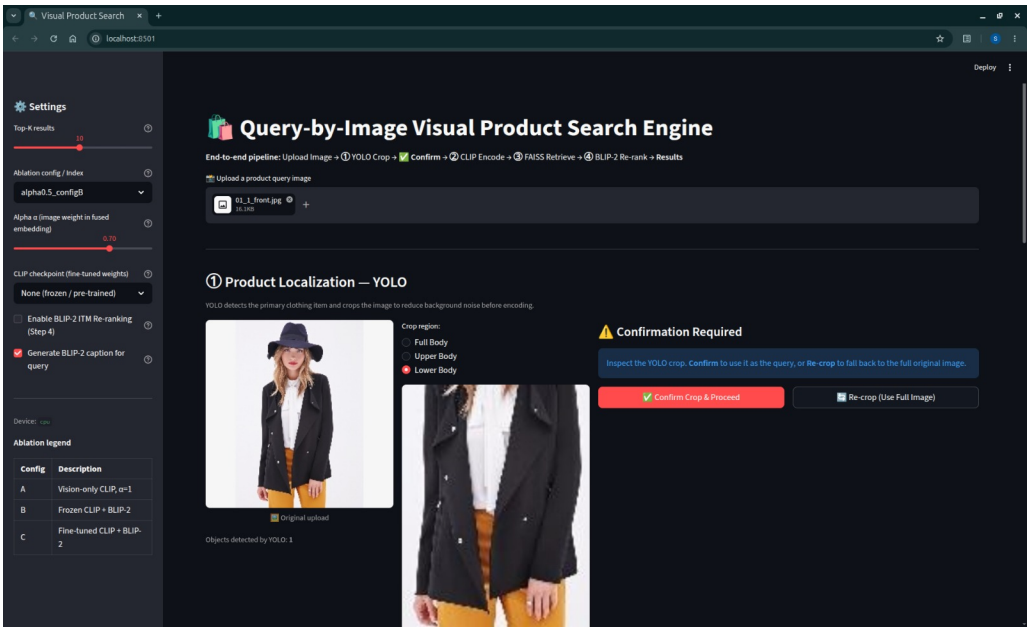

Figure 2: Screenshot of query upload interface.

10.2 Step 2: Retrieval Pipeline

The uploaded image is processed using:

1. YOLOv8 localization
2. Fine-tuned CLIP embedding generation
3. FAISS retrieval
4. BLIP reranking

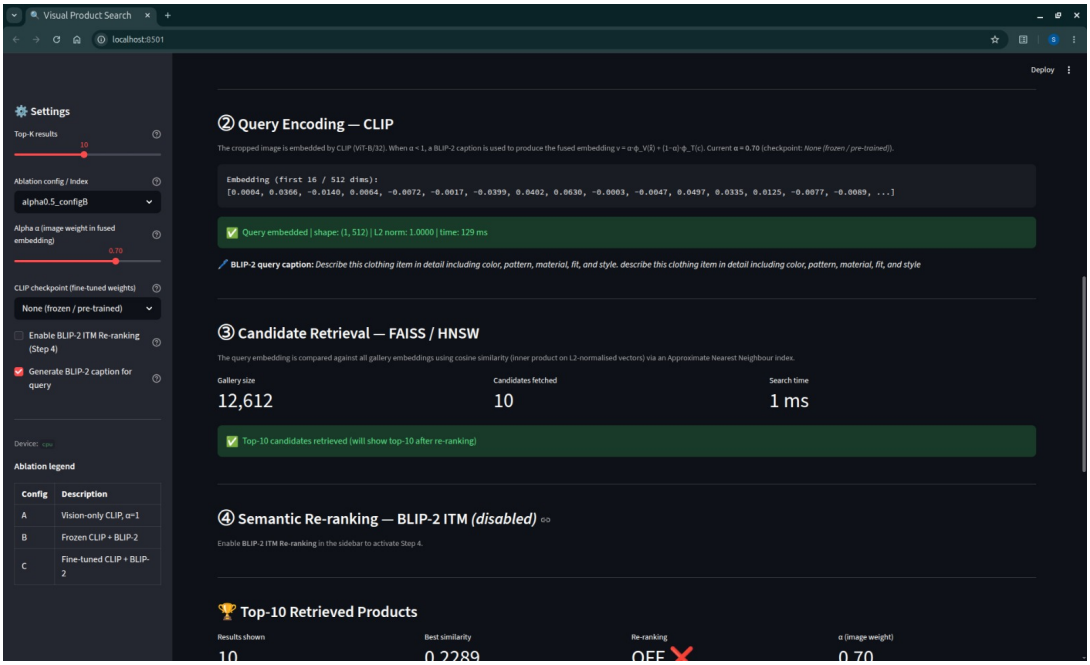

Figure 3: Screenshot showing retrieval pipeline or detected crops.

10.3 Step 3: Top-K Retrieval Results

The system displays the top-K retrieved products along with similarity scores.

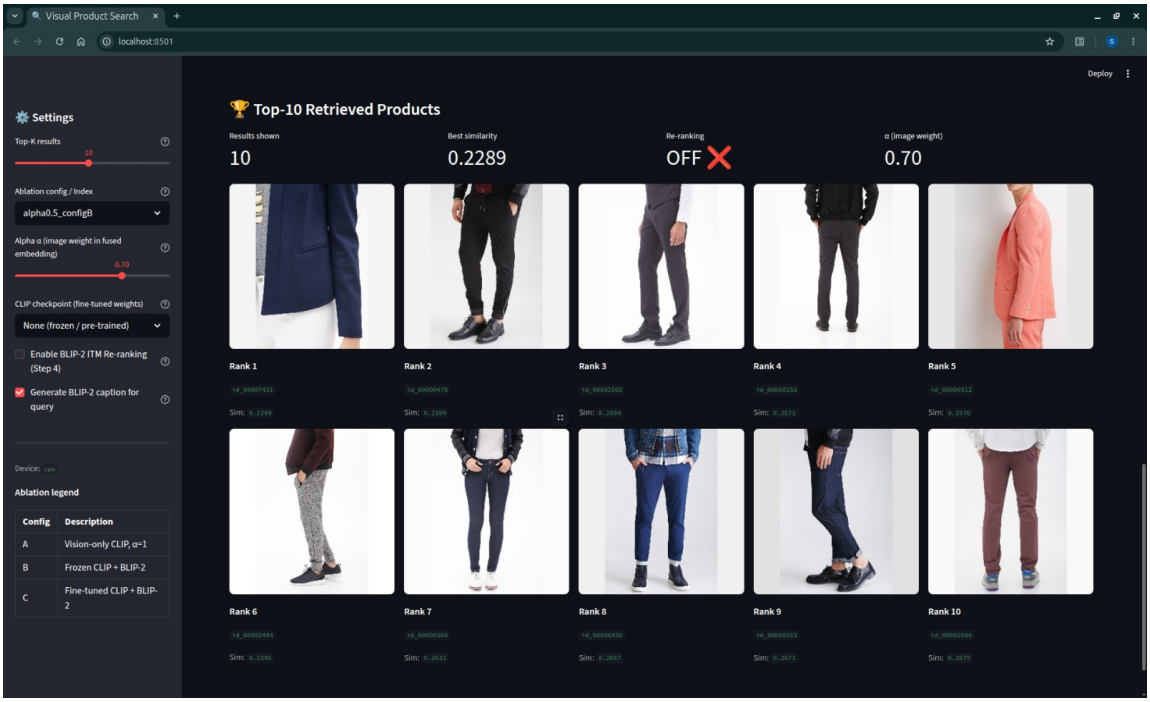

Figure 4: Screenshot of top-K retrieval results.

11 Runtime Observations

All experiments were conducted using Kaggle GPU notebooks with NVIDIA T4 GPUs. Some practical observations from the implementation are:

- FAISS retrieval is extremely fast once embeddings are indexed
- BLIP-2 reranking improves retrieval quality but increases inference time
- YOLO-based cropping significantly reduces background noise
- Fine-tuning CLIP gives the largest overall performance improvement

The final system remains suitable for interactive retrieval despite the additional reranking stage.

12 Limitations

Although the system performs well overall, several limitations still remain.

- BLIP-2 reranking is computationally expensive
- Retrieval quality can degrade for very cluttered query images
- Some clothing categories remain visually ambiguous
- The current system focuses only on fashion retrieval

Future improvements could include:

- Lightweight reranking models
- Multi-scale retrieval
- Better attribute-aware embeddings
- Larger retrieval benchmarks

13 Conclusion

In this project, we developed a complete visual product retrieval system for fashion search.

The system combines:

- YOLOv8 for localization
- BLIP-2 for semantic understanding
- Fine-tuned CLIP embeddings
- FAISS HNSW for efficient retrieval

Our experiments show that fine-tuning CLIP is the most important factor for improving retrieval quality. BLIP-based semantic fusion further improves ranking consistency, especially when using fusion weight $\alpha = 0.7$.

The final system successfully supports both offline batch evaluation and online interactive retrieval, making it practical for real-world fashion search applications.